Zombi Escape

Diagrama de clases UML · Proyecto programado II · Programación Orientada a Objetos · TEC

Contenido

Diagrama	Paquetes	Qué muestra
1. Modelo y lógica del juego	`modelo`, `logica`	Jugadores (herencia Jugador → Superviviente / Sniper), zombies, mapa, la Partida con sus reglas, un hilo por zombie y la carga validada de escenarios.
2. Red: cliente y servidor	`red.Servidor`, `red.Cliente`, `red.Models`	Servidor con un hilo de lectura y un hilo de envío por cliente, protocolo de mensajes y la "foto" EstadoJuego que viaja por el socket.
3. Interfaz gráfica	`gui`, `red.Cliente`	Arranque (servidor o jugador), login con portada animada, ventana de juego, panel del mapa y el hilo que envía los movimientos.

Las clases que ya aparecen completas en otro diagrama se muestran solo con su nombre y la nota «ver diagrama N».

Notación

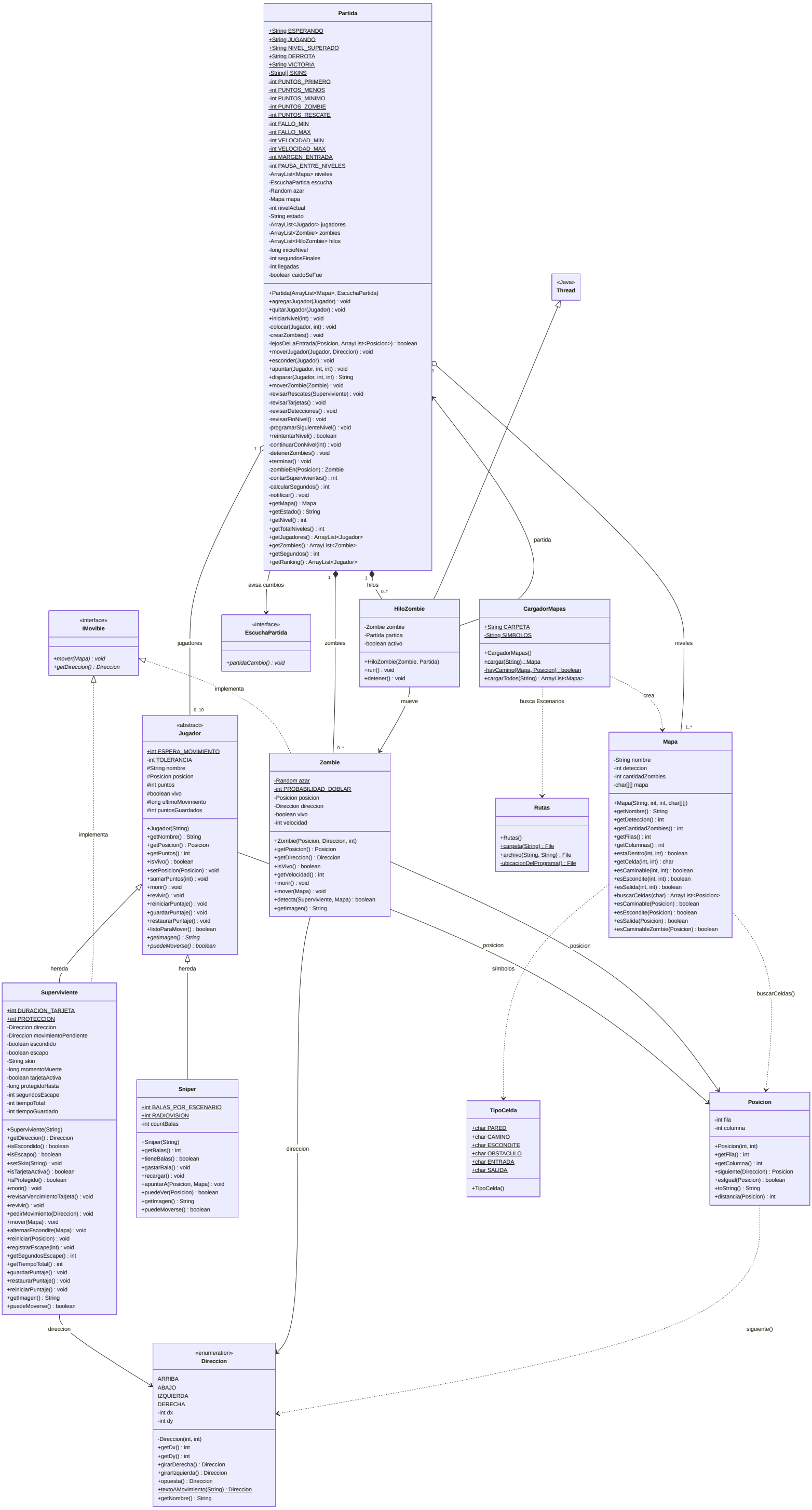
Símbolo	Significado
`+` `-` `#` `~`	Visibilidad: público, privado, protegido, de paquete
subrayado	Atributo o método estático (constantes)
cursiva	Método abstracto (debe implementarlo la clase hija)
Flecha con triángulo, línea continua	Herencia (`extends`)
Flecha con triángulo, línea punteada	Implementa una interfaz (`implements`)
Rombo negro	Composición: la parte no existe sin el todo (Partida crea y destruye sus zombies e hilos)
Rombo blanco	Agregación: el todo usa partes que existen por su cuenta (Partida y sus jugadores)
Flecha continua	Asociación: la clase guarda una referencia a la otra
Flecha punteada	Dependencia: la usa o la crea, sin guardarla
«interface», «abstract», «enumeration»	Tipo especial de clase
«Java»	Clase de la biblioteca de Java (Thread, JFrame, JPanel)

Decisiones de diseño principales

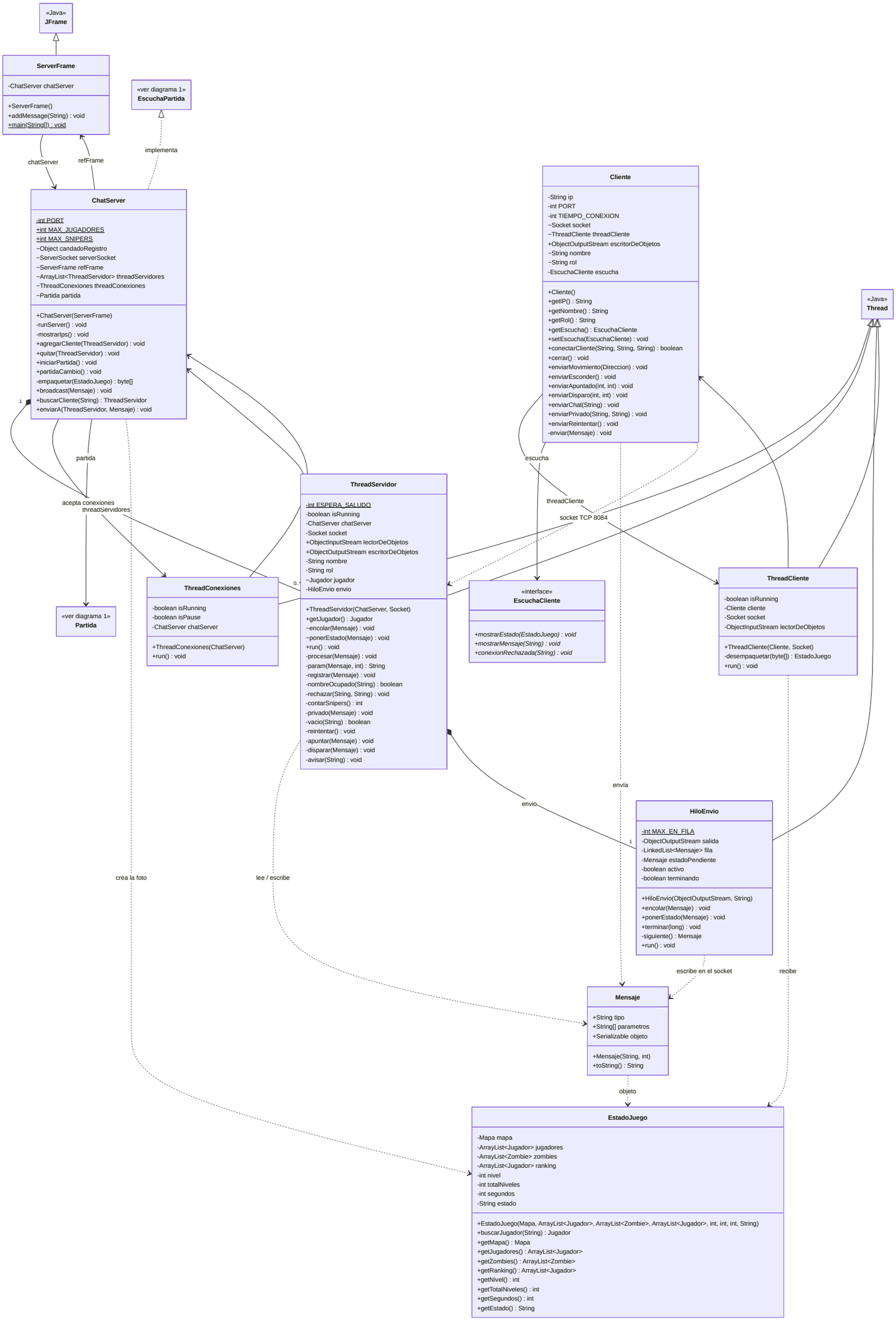
Herencia y polimorfismo	`Jugador` es abstracta; `getImagen()` y `puedeMoverse()` los define cada hija, así la ventana y la partida no necesitan `instanceof`.
Interfaz IMovible	La comparten `Superviviente` y `Zombie`, que se mueven por el mapa aunque no son el mismo tipo de objeto.
Observador	`Partida` avisa los cambios por `EscuchaPartida` (la implementa el servidor) y el cliente avisa a la ventana por `EscuchaCliente`. La lógica no depende de la red ni de la interfaz.
Hilos	Un `HiloZombie` por zombie; por cada cliente, un `ThreadServidor` que lee y un `HiloEnvio` que escribe; `HiloMovimiento` envía los pasos a ritmo fijo sin congelar la ventana.
Un cliente lento no frena a nadie	`HiloEnvio` es el único que escribe en su socket: la partida solo le deja el mensaje. Del estado se envía siempre la foto más reciente, empaquetada una sola vez para todos.
Servidor robusto	Cada conexión se limpia en un `finally`; los nombres son únicos; los límites (10 jugadores, 2 snipers) se revisan con un candado; una conexión que no se presenta se cierra a los 10 s.
Escenarios validados	`CargadorMapas` revisa símbolos, números y, con una búsqueda por anchura (BFS), que cada entrada tenga camino a una salida.
Servidor autoritativo	El cliente solo pide (MOVE, SHOOT, AIM, HIDE); el servidor valida con `Partida` y envía a todos un `EstadoJuego`.

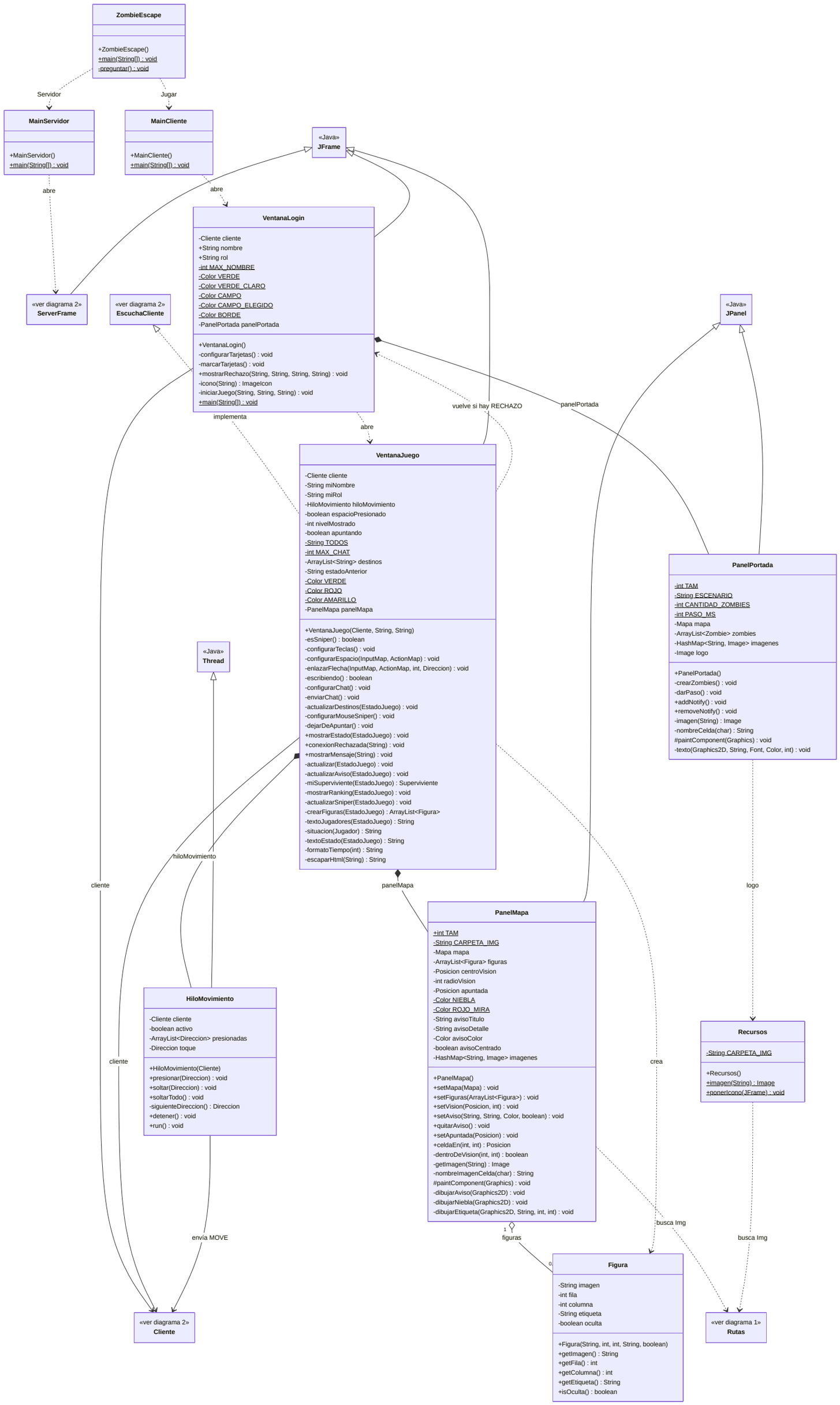
Generado a partir del código fuente compilado (javap), por lo que los atributos y métodos coinciden con el código entregado. Se omiten solo los componentes visuales generados por NetBeans (botones, etiquetas); todas las clases del proyecto aparecen en algún diagrama.

Modelo y lógica del juego



Red: cliente y servidor (sockets)




ZombieEscape
+ZombieEscape()
+main(String[]) : void
-preguntar() : void
Servidor
Jugar
MainServidor
+MainServidor()
+main(String[]) : void
MainCliente
+MainCliente()
+main(String[]) : void
«Java»
JFrame
abre
abre
VentanaLogin
-Cliente cliente
+String nombre
+String rol
-int MAX_NOMBRE
-Color VERDE
-Color VERDE_CLARO
-Color CAMPO
-Color CAMPO_ELEGIDO
-Color BORDE
-PanelPortada panelPortada
+VentanaLogin()
-configurarTarjetas() : void
-marcarTarjetas() : void
+mostrarRechazo(String, String, String, String) : void
-icono(String) : ImageIcon
-iniciarJuego(String, String, String) : void
+main(String[]) : void
«ver diagrama 2»
ServerFrame
«ver diagrama 2»
EscuchaCliente
«Java»
JPanel
vuelve si hay RECHAZO
panelPortada
implementa
abre
VentanaJuego
-Cliente cliente
-String miNombre
-String miRol
-HiloMovimiento hiloMovimiento
-boolean espacioPresionado
-int nivelMostrado
-boolean apuntando
-String TODOS
-int MAX_CHAT
-ArrayList<String> destinos
-String estadoAnterior
-Color VERDE
-Color ROJO
-Color AMARILLO
-PanelMapa panelMapa
+VentanaJuego(Cliente, String, String)
-esSniper() : boolean
-configurarTeclas() : void
-configurarEspacio(InputMap, ActionMap) : void
-enlazarFlecha(InputMap, ActionMap, int, Direccion) : void
-escribiendo() : boolean
-configurarChat() : void
-enviarChat() : void
-actualizarDestinos(EstadoJuego) : void
-configurarMouseSniper() : void
-dejarDeApuntar() : void
+mostrarEstado(EstadoJuego) : void
+conexionRechazada(String) : void
+mostrarMensaje(String) : void
-actualizar(EstadoJuego) : void
-actualizarAviso(EstadoJuego) : void
-miSuperviviente(EstadoJuego) : Superviviente
-mostrarRanking(EstadoJuego) : void
-actualizarSniper(EstadoJuego) : void
-crearFiguras(EstadoJuego) : ArrayList<Figura>
-textoJugadores(EstadoJuego) : String
-situacion(Jugador) : String
-textoEstado(EstadoJuego) : String
-formatoTiempo(int) : String
-escaparHtml(String) : String
PanelPortada
-int TAM
-String ESCENARIO
-int CANTIDAD_ZOMBIES
-int PASO_MS
-Mapa mapa
-ArrayList<Zombie> zombies
-HashMap<String, Image> imagenes
-Image logo
+PanelPortada()
-crearZombies() : void
-darPaso() : void
+addNotify() : void
+removeNotify() : void
-imagen(String) : Image
-nombreCelda(char) : String
#paintComponent(Graphics) : void
-texto(Graphics2D, String, Font, Color, int) : void
«Java»
Thread
hiloMovimiento
cliente
panelMapa
logo
PanelMapa
+int TAM
-String CARPETA_IMG
-Mapa mapa
-ArrayList<Figura> figuras
-Posicion centroVision
-int radioVision
-Posicion apuntada
-Color NIEBLA
-Color ROJO_MIRA
-String avisoTitulo
-String avisoDetalle
-Color avisoColor
-boolean avisoCentrado
-HashMap<String, Image> imagenes
+PanelMapa()
+setMapa(Mapa) : void
+setFiguras(ArrayList<Figura>) : void
+setVision(Posicion, int) : void
+setAviso(String, String, Color, boolean) : void
+quitarAviso() : void
+setApuntada(Posicion) : void
+celdaEn(int, int) : Posicion
-dentroDeVision(int, int) : boolean
-getImagen(String) : Image
-nombreImagenCelda(char) : String
#paintComponent(Graphics) : void
-dibujarAviso(Graphics2D) : void
-dibujarNiebla(Graphics2D) : void
-dibujarEtiqueta(Graphics2D, String, int, int) : void
Recursos
-String CARPETA_IMG
+Recursos()
+imagen(String) : Image
+ponerIcono(JFrame) : void
HiloMovimiento
-Cliente cliente
-boolean activo
-ArrayList<Direccion> presionadas
-Direccion toque
+HiloMovimiento(Cliente)
+presionar(Direccion) : void
+soltar(Direccion) : void
+soltarTodo() : void
-siguienteDireccion() : Direccion
+detener() : void
+run() : void
cliente
crea
busca Img
busca Img
envía MOVE
1
figuras
0..
Figura
-String imagen
-int fila
-int columna
-String etiqueta
-boolean oculta
+Figura(String, int, int, String, boolean)
+getImagen() : String
+getFila() : int
+getColumna() : int
+getEtiqueta() : String
+isOculta() : boolean
«ver diagrama 2»
Cliente
«ver diagrama 1»
Rutas